

Arquitectura de Launchers de Minecraft

Especificación Técnica: Autenticación Híbrida (Premium / No-Premium) y Sistema Global de Cosméticos

Este documento proporciona un análisis arquitectónico exhaustivo para el desarrollo de un launcher personalizado de Minecraft. Se detalla el flujo de autenticación, la validación de identidad a través de servicios de terceros y propios, y la inyección síncrona/asíncrona de cosméticos e inventarios mediante una API construida en **ASP.NET Core** con persistencia en bases de datos relacionales.

1. Fundamentos: ¿Cómo opera un Launcher de Minecraft?

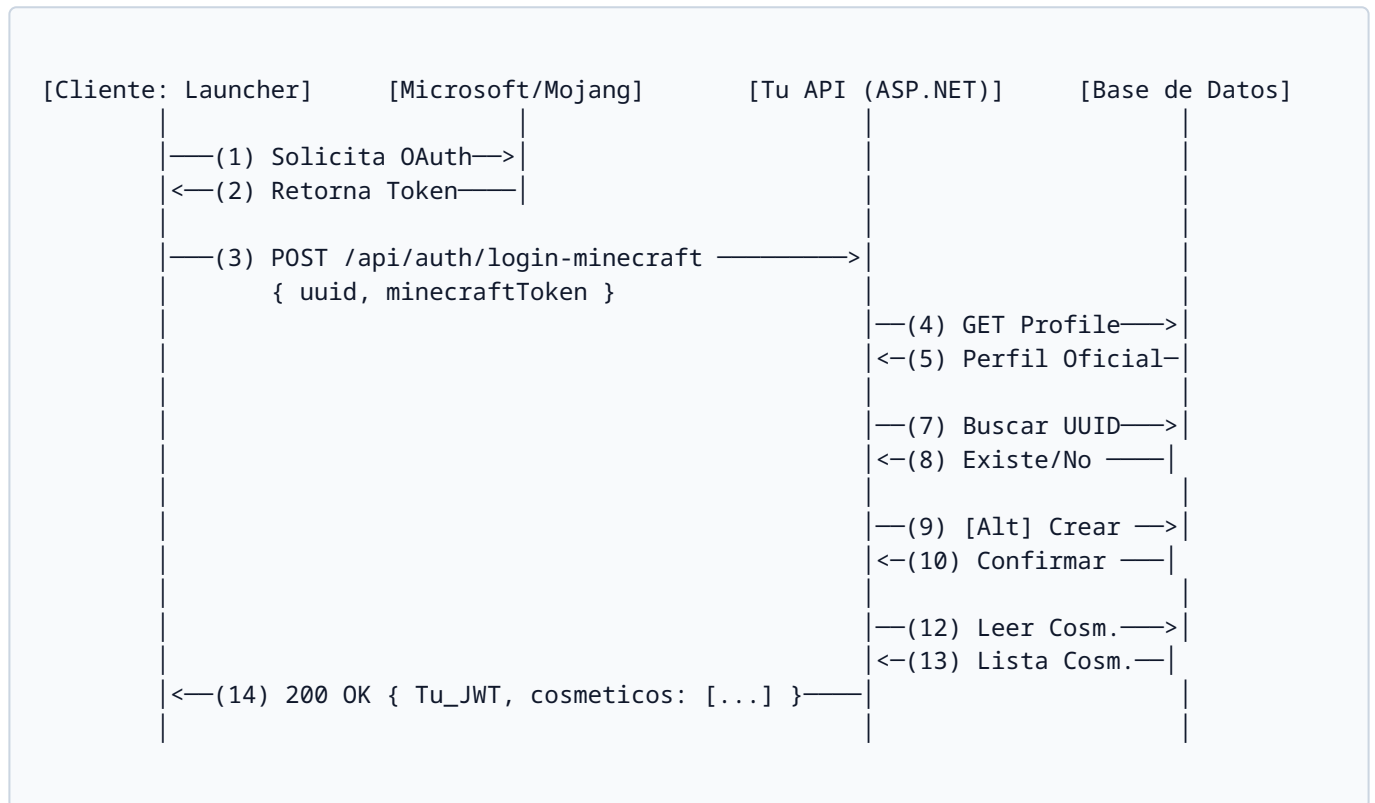
Un launcher personalizado actúa exclusivamente como un orquestador del entorno de ejecución antes de la inicialización de la Máquina Virtual de Java (JVM). Sus tres responsabilidades nucleares en el cliente son:

1. **Resolución de Dependencias y Assets:** Descarga de archivos binarios estructurales (.jar de la versión del juego), librerías nativas del sistema operativo (.dll, .so, .dylib) y recursos estáticos indexados por los manifiestos oficiales de Mojang.
2. **Aduana de Identidad (Autenticación):** Intercambio seguro de credenciales para la obtención de un identificador único global (UUID) y un token criptográfico de sesión.
3. **Construcción del ClassPath y Runtime:** Ensamblado dinámico del comando de ejecución CLI para la JVM, inyectando variables de entorno, asignación de memoria RAM y los argumentos nativos que el motor de Minecraft requiere para inicializarse.

2. Análisis del Flujo de Autenticación Premium

El flujo diseñado para usuarios Premium delega la validación de credenciales a Microsoft/Mojang, pero centraliza la sesión del ecosistema propio mediante un mecanismo de doble validación (Backend-to-Backend).

Diagrama de Secuencia: Flujo Premium



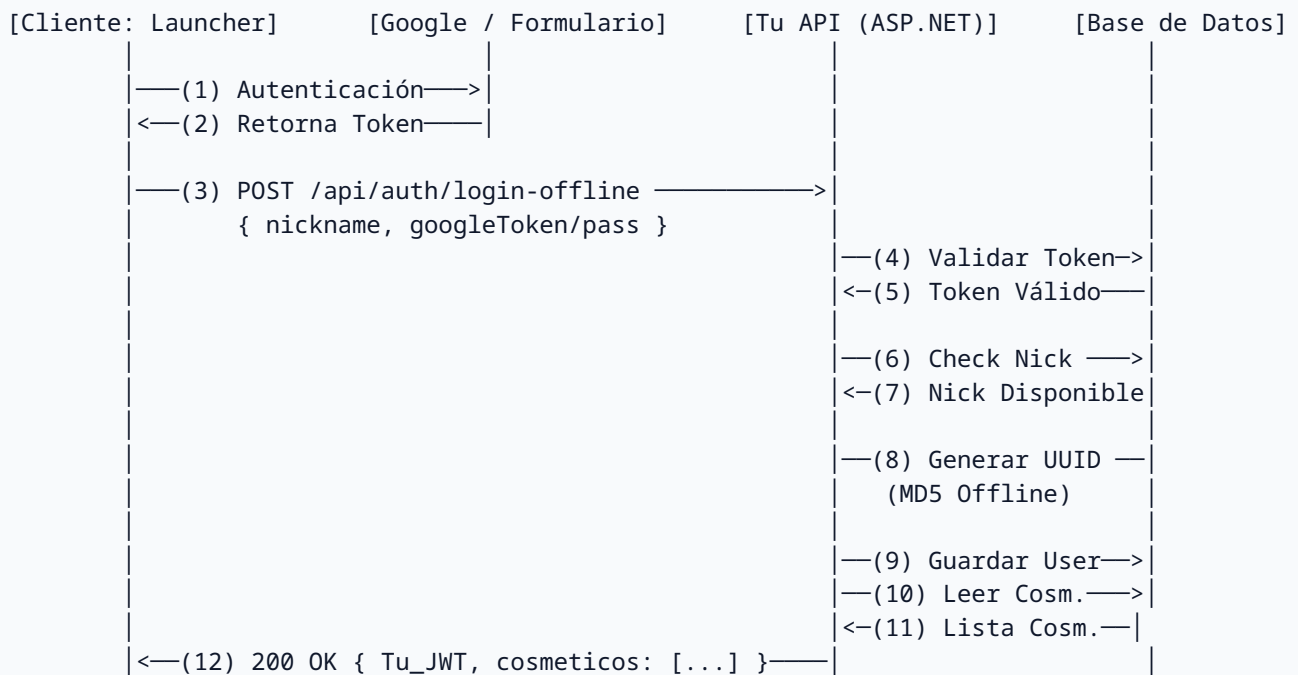
Explicación Detallada del Proceso Premium

- **Fase de Validación Local (Pasos 1-2):** El launcher interactúa con los servicios de identidad de Microsoft vía OAuth2 y XSTS, obteniendo un `minecraftToken` de corta duración y el UUID verificado del jugador.
- **Validación e Inmunidad a Suplantaciones (Pasos 3-6):** El launcher envía el token a tu API de ASP.NET Core. **Crítico:** Tu API no confía en el UUID provisto por el cliente; en su lugar, consume directamente el endpoint oficial de Mojang (`minecraftservices.com`) actuando como proxy de validación. Si el token es inválido o el UUID fue alterado en el cliente, la API corta el flujo inmediatamente con un error 401 `Unauthorized`.
- **Persistencia y Sincronización (Pasos 7-10):** Una vez comprobada la autenticidad, la API consulta la existencia del UUID en tu base de datos. Si es la primera interacción del jugador en tu ecosistema, se realiza un registro automático (*Upsert*) resguardando la integridad mediante índices únicos.
- **Sesión del Ecosistema Propio (Pasos 11-14):** La API genera un token propio (`Tu_JWT`) firmado simétricamente (HMAC-SHA256), consulta el inventario de cosméticos adquiridos por el usuario en la base de datos y emite una respuesta unificada con los recursos listos para ser consumidos.

3. Arquitectura del Flujo No-Premium (Offline)

En el escenario No-Premium, tu infraestructura asume el rol completo de Proveedor de Identidad (IdP). Ya no existe una entidad externa (Mojang) para validar al usuario, por lo que los mecanismos de protección contra colisiones de nombres e inyección de sesiones se vuelven críticos.

Diagrama de Secuencia: Flujo No-Premium / Google



Explicación Detallada del Proceso No-Premium

- **Autenticación de Cuenta (Pasos 1-5):** El usuario utiliza una cuenta federada (Google OAuth2) o un sistema nativo de correo y contraseña controlado por tu backend. Tu API valida criptográficamente el token de Google o verifica el hash de la contraseña (utilizando algoritmos como BCrypt o Argon2id).
- **Generación Determinista de Identidad (Pasos 6-8):** Al no existir un UUID de Mojang, tu API debe calcular el UUID offline bajo el estándar oficial de Minecraft. La fórmula matemática determinista es: $UUID_{offline} = MD5("OfflinePlayer:" + nickname)$. De este modo, el UUID siempre corresponderá de manera unívoca al nombre del jugador dentro del juego.
- **Control de Colisiones (Paso 6):** Antes de persistir, la API verifica que el nickname solicitado no pertenezca a una cuenta Premium ya registrada en tu base de datos ni a otro usuario offline con credenciales distintas, evitando el secuestro de identidades visuales dentro de tu red.
- **Unificación de Salida:** Al igual que en el flujo premium, la API responde con Tu_JWT y los cosméticos asociados a ese perfil offline. El comportamiento para el Launcher a partir de este punto se estandariza.

4. Convivencia en Base de Datos e Inyección del Argumento de Lanzamiento

Para soportar ambos tipos de cuenta de manera transparente sin corromper la integridad referencial, el diseño de la base de datos debe implementar una tipificación explícita de cuentas y mapear el origen del proveedor.

Campo	Tipo de Datos	Restricción	Descripción
Id	INT / UUID	Primary Key	Identificador interno de la base de datos.
MinecraftUUID	VARCHAR(36)	Unique Index	UUID Oficial de Mojang o UUID generado vía MD5 Offline.
Username	VARCHAR(16)	Unique (Local)	Nickname del jugador visible dentro de las partidas.
AccountType	ENUM / INT	Not Null	Diferencia explícitamente entre Premium y NoPremium.
ProviderId	VARCHAR(255)	Nullable	ID de sujeción (Microsoft ID, Google Subject ID o Hash).

Estrategia del Comando CLI (Java Execution):

Al ejecutar el binario del juego, el launcher altera los argumentos de línea de comandos según el tipo de cuenta:

- **Premium:** `--username "Nick" --uuid "UUID_Real" --accessToken "Token_Mojang"`
- **No-Premium:** `--username "Nick" --uuid "UUID_Offline" --accessToken "null"`

5. Suministro Dinámico de Cosméticos en el Cliente

Dado que el juego en modo offline no consulta las texturas en los servidores de Mojang (restando la skin por defecto de Steve/Alex), y que el juego base carece de soporte para renderizar capas o accesorios de terceros (alas, mascotas), la arquitectura requiere una extensión en el motor del cliente.

El Ciclo de Renderizado del Mod / Agent Java

Independientemente de cómo inicie sesión el usuario, el launcher deposita o inyecta un componente de código dentro del ciclo de Minecraft (desarrollado sobre la API de **Fabric** o **Forge**). Las fases de este componente son:

1. **Gancho de Renderizado (Render Hook):** El mod intercepta el evento nativo donde el motor gráfico de Minecraft (utilizando OpenGL/LWJGL) se dispone a dibujar la entidad de un jugador (`AbstractClientPlayerEntity`) en pantalla.
2. **Consulta Asíncrona a tu API:** Cuando un jugador entra en el rango de visión del cliente, el mod extrae su UUID y realiza una llamada HTTP asíncrona no bloqueante hacia tu API: `GET /api/cosmetics/active/{uuid}`.
3. **Validación de Sesión del Propietario:** Para evitar alteraciones locales en la memoria del cliente (donde un usuario No-Premium podría hackear el mod para visualizar cosméticos que no posee), tu API valida la propiedad real del ítem cruzando la consulta con la base de datos de compras.
4. **Caché y Enlazado de Texturas (Texture Binding):** Tras recibir el JSON descriptivo de los cosméticos activos (URLs de imágenes .png o modelos 3D .glTF alojados en tu CDN/S3), el mod descarga los

archivos en segundo plano, inyecta la textura en el manejador de texturas nativo de Minecraft y renderiza el cosmético acoplado a la rotación de los huesos del modelo del jugador.

A través de este desacoplamiento técnico, tu API en ASP.NET Core se consolida como una infraestructura agnóstica de alto rendimiento, capaz de monetizar, autenticar y dotar de identidad visual tanto a la comunidad premium como a la comunidad offline bajo una única base de código unificada.